

FinResearch AI

Multi-Agent Financial Research System

Complete Project Documentation

Written for first-time readers: what it is, how it works, and why every choice was made.

Companion to: PROJECT_PRESENTATION.pdf (the evaluation deck)

October 2026

All figures in demo data are illustrative samples prepared for product demonstration.

Contents

1. What is FinResearch AI? (problem, solution, audience)
2. Complete feature tour
3. Technology stack: what is used where, and why
4. System architecture and data flow
5. The seven agents (logic, code, files) + orchestrator
6. Data model (12 tables)
7. API reference (20 routes)
8. Website guide (every page)
9. Setup: local development and cloud deployment
10. Security, reliability and graceful degradation
11. Testing
12. Troubleshooting
13. Limitations and future roadmap
14. Glossary for beginners

1. What is FinResearch AI?

1.1 The problem it solves

Analysing a company the professional way means reading 100+ page annual reports, copying numbers into spreadsheets, checking for warning signs in the fine print, comparing several companies, and writing a summary. A single deep review can take hours. Most AI chat tools can summarize a document in seconds, but they invent numbers, cannot show where a figure came from, and forget everything between uploads.

1.2 The solution

FinResearch AI is a full-stack web application where a user uploads financial documents (PDF, DOCX or plain text) for one or more companies. The system automatically reads the documents, extracts 26 standard financial metrics, flags risks with exact source quotes, optionally builds a semantic search index, and then lets the user ask research questions, compare companies, and generate a polished analysis report - with every claim traceable back to a specific document line.

1.3 The three design promises

- Traceable: every metric and every risk stores the source text it came from. Answers carry [S1], [S2] style citations with document name, section and excerpt.
- Honest: deterministic code (regular expressions and maths) does the precision work; AI does the language work. If AI is unavailable, every feature degrades to a rule-based mode instead of failing.
- Runnable on a student budget: the entire stack runs on free tiers (Render web service, Neon Postgres, Groq AI) and the code keeps optional features (like embeddings) strictly optional.

1.4 Who it is for

- Students and analysts who want a first-pass view of a company fast.
- Investors comparing two or three companies on the same metrics.
- Anyone who wants AI help WITHOUT losing the evidence trail.

2. Complete feature tour

Feature	What it does	Where to see it
Accounts	Register and log in (bcrypt-hashed passwords, secure session cookies). Every user sees only their own data.	/register, /login
Document upload	Drag-and-drop PDF/DOCX/TXT with company name, ticker, fiscal year and document type. Files are parsed and processed by a background pipeline.	Dashboard -> Documents -> Upload
Agent pipeline	Document Agent (index) -> Extraction Agent (metrics) -> Red Flag Agent (risks) -> Embedding Agent (optional vectors). Each step logs to an activity trail.	Document detail page
Metrics	26 standard metrics per document (revenue, margins, EPS, ratios, cash flow...) each with the source line used as evidence.	Metrics page, document detail
Risk flags	Deterministic red-flag detection (auditor issues, going concern, accounting control, liquidity) plus metric anomalies and AI-added findings, all with quotes.	Risks page
Cross-period trends	When two years of the same company exist, the system compares them and raises trend risks (declining margins, rising debt, negative FCF...).	Risks page
Semantic search (optional)	Embedding Agent stores vector embeddings so research can use meaning-based retrieval. Without a provider it falls back to keyword search automatically.	Document detail -> Build embeddings
Research Q&A	Ask questions in plain English. The Research Agent decomposes compound questions, retrieves evidence, and answers with [S1] citations. It refuses to invent numbers.	Research page (chat)
Benchmark	Compare up to several companies on revenue, profitability, leverage, liquidity and risk counts, with period-mismatch warnings.	Benchmark page
Reports	Generate a structured analysis report (Key Financials table + narrative sections + disclaimer). Falls back to a deterministic report if AI fails. Download as text.	Reports page
Sessions	Group documents into research sessions and scope questions to a session's documents.	Sessions page
Health	Public /api/health endpoint describing AI configuration and agent status - great for live debugging demos.	/api/health

3. Technology stack: what is used where, and why

3.1 Frontend

Technology	Where / for what	Why this one	Why not the alternative
Next.js 16 (App Router)	The entire web app: pages, layouts, API routes	One codebase, server rendering, fast deploys, file-based routing	Plain React (Vite) needs a separate backend server; PHP/Laravel lacks the React ecosystem
React 19	All interactive UI (uploads, chat, tables, charts)	Component model, hooks, huge ecosystem	Vanilla JS becomes unmanageable at this UI complexity
TypeScript	Every source file	Catches type errors before runtime; safer refactors	Plain JS lets small mistakes reach production
Tailwind CSS 4	All styling	Utility classes = fast, consistent, tiny CSS	Bootstrap/MUI: heavier bundle, fighting the framework to customise
Recharts	Dashboard and benchmark charts	Declarative React charts	Chart.js: imperative canvas, awkward with React state
Framer Motion	Page/panel animations	Declarative motion in React	CSS transitions get clumsy for sequenced UI motion
react-dropzone	Upload panel	Accessible drag-and-drop with validation hooks	Hand-rolled DnD is error-prone (browser quirks)
react-hot-toast	Notifications	Lightweight toasts for agent status	Full alert libraries are overkill

3.2 Backend and data

Technology	Where / for what	Why this one	Why not the alternative
Next.js API Routes (Node)	20 REST endpoints under /api	Same deploy unit as the UI; no CORS issues	Separate Express/Django service = 2x deploy, auth sharing pain
PostgreSQL (Neon)	Primary database (12 tables)	Relational fit for financial data; SQL aggregation for benchmarks; ACID; free serverless tier	MongoDB: weak joins/aggregation for company-period analytics; no real need for document flexibility
Drizzle ORM 0.45	All queries + migrations	TypeScript-first, SQL-like, light; schema is code	Prisma: heavier runtime and codegen; raw SQL: injection risk and no type checks
JSONB vectors	document_chunks.embedding	Postgres JSONB stores vectors with zero extensions; cosine similarity computed in the app - fine to medium scale	pgvector: better at large scale but needs extension support and index tuning (a roadmap item)
NextAuth (credentials)	Register/login/sessions	Battle-tested, HTTP-only cookies, no third-party lock-in	Clerk/Auth0: paid at scale; rolling your own JWT auth is a security risk
bcryptjs	Password hashing	Pure JS (no native build issues on Render); salted slow hashes	Plain SHA-256 is too fast to brute-force safely; native bcrypt breaks on some hosts
Groq (openai/gpt-oss-120b)	All AI text work	Generous free tier, very fast LPU inference, OpenAI-compatible API	OpenAI: paid; Gemini: slower and its key errors are confusing (this project once mis-labelled them); local LLMs: need a GPU server
unpdf + pdf-parse	PDF text extraction (two parsers, one as fallback)	unpdf is bundler-safe; pdf-parse proven; both are lightweight	Puppeteer/Playwright: headless Chromium needs hundreds of MB of RAM that free tiers lack

Technology	Where / for what	Why this one	Why not the alternative
mammoth	DOCX text extraction	Clean, pure-JS .docx reader	LibreOffice conversion needs heavy system packages

3.3 AI architecture choices (the interesting part)

Decision	Why it is done this way	The alternative we rejected
Hybrid deterministic + AI	Numbers must be exact, so metrics and red flags use regex/math first; AI is used for language (narrative, synthesis) and as an ADDITIONAL reviewer, never the only truth	Pure-AI extraction hallucinates numbers; pure-rules cannot write narrative
Custom agents, not LangChain	Each agent is a small, readable TypeScript file with explicit inputs/outputs and its own fallback - easy to explain, test and debug	LangChain adds abstraction layers and dependencies that hide the flow and bloat the bundle
Provider-agnostic embeddings	EMBEDDING_BASE_URL/ EMBEDDING_MODEL/ EMBEDDING_API_KEY point at any OpenAI-compatible endpoint (Gemini, Jina, Ollama...)	Hard-coding one vendor would lock the project and fail when that vendor is down
Grounded answers only	The Research Agent validates that numbers in its answer appear in retrieved source text; on any failure it returns verbatim evidence passages instead of guessing	Trusting the LLM output blindly is how AI products lose credibility
Deterministic report skeleton	The Key Financials table in reports is rendered from stored metrics by code, not by the LLM	Letting the LLM re-type numbers invites transcription errors

3.4 DevOps

Technology	Where / for what	Why this one	Why not the alternative
Render (free web service)	Hosts the Next.js app (build: npm ci && npm run build, start: npm start)	Free tier, GitHub auto-deploy, env var UI, easy for students	Vercel: hobby limits and function quirks for this workload; a VPS costs money and needs maintenance
Neon Postgres	Serverless Postgres with pooled + direct URLs	Free tier, autosuspend, connection pooling built in	Local/RDS Postgres: always-on cost or not reachable from the cloud
Render Cron Job	Pings /api/worker/documents every 15 minutes with a secret	Drains the background job queue reliably on a schedule	Queue services (SQS, BullMQ/Redis): extra accounts, extra cost, extra failure points
GitHub (git)	Source control, PR reviews, deployment trigger	Industry standard; PR workflow keeps main stable	Emailing zip files / shared drives is not version control
Node 22 (.nvmrc)	Pinned runtime version	Reproducible builds local and on Render	Floating default Node versions cause 'works on my machine' bugs

4. System architecture and data flow

4.1 The big picture

A single Next.js application serves both the UI and the API. The browser never talks to the database or the AI directly - all secrets stay on the server. Documents are processed by an asynchronous pipeline (a job queue in Postgres) so uploads stay fast and never time out.

- Browser (React) -- HTTP --> Next.js pages and API routes
- API routes -- Drizzle ORM --> PostgreSQL (Neon): users, companies, documents, chunks, metrics, risks, reports, chats, benchmarks, agent_logs, jobs
- Agents -- HTTPS --> Groq (chat/completions, JSON mode) and an optional embeddings provider
- Upload creates a 'processing job'; the /api/worker/documents endpoint (cron every 15 min, or immediate kick) claims jobs atomically and runs the pipeline

4.2 Upload-time pipeline (runs once per document)

- 1. POST /api/documents validates auth, file type (pdf/docx/txt) and size, extracts text (two PDF parsers with fallback), creates the document row and enqueues a job.
- 2. Pipeline Orchestrator runs stages in order with failure isolation: Document -> Extraction -> Red Flag -> Embedding (optional).
- 3. A 'partial' result is allowed: if one stage fails, later stages still run and the document status explains what failed.
- 4. Every stage writes to agent_logs (agent name, action, status, duration, details) - the visible activity trail on the document page.

4.3 On-demand agents (run when the user asks)

- Research Agent: question -> decomposition -> retrieval -> grounded answer with citations (POST /api/chat).
- Benchmark Agent: company set -> metric profiles -> comparisons + narrative (GET/POST /api/benchmark).
- Report Agent: company profile + metrics + risks -> structured markdown report (POST /api/reports).

4.4 Resilience design

- Job queue: atomic claim (UPDATE ... WHERE status='queued'), 15-minute stale-lock expiry, retry with backoff up to max attempts, batch size 5 per worker run.
- AI failures: every agent has a deterministic fallback (regex extraction, keyword search, local report, local benchmark).
- Embeddings are never required: no provider -> 'unavailable' status and keyword retrieval keeps research fully working.

5. The seven agents (logic, code, files)

Agent files live in src/lib/agents/. The orchestrator (orchestrator.ts) runs upload-time agents; research/report/benchmark agents run on demand. llmClient.ts is the single door to the Groq API; aiStatus.ts tracks configuration and converts raw AI errors into user-safe messages.

5.1 Document Agent - documentAgent.ts

Job: turn raw text into searchable chunks. It splits text into overlapping chunks of 1500 characters (200 char overlap), detects section headings (BUSINESS OVERVIEW, RISK FACTORS, MANAGEMENT DISCUSSION, FINANCIAL STATEMENTS, LIQUIDITY, RESULTS OF OPERATIONS, NOTES) with regular expressions, stores each chunk in document_chunks with its section and position, and exposes hybrid search (keyword relevance + optional cosine similarity over embeddings).

Why 1500/200: a chunk must be small enough to quote precisely in an answer, but large enough to keep context; the overlap ensures a sentence that straddles a boundary exists complete in at least one chunk.

```
export function chunkText(text: string, chunkSize = 1500, overlap = 200): string[] {
  const chunks: string[] = [];
  let start = 0;
  while (start < text.length) {
    const end = Math.min(start + chunkSize, text.length);
    chunks.push(text.slice(start, end));
    start += chunkSize - overlap;
  }
  return chunks;
}
```

Key functions: chunkText, extractSections, processDocument, searchDocumentChunks, searchDocumentCollection.

5.2 Extraction Agent - extractionAgent.ts + extractionUtils.ts

Job: extract 26 financial metrics. It uses a hybrid strategy: (1) deterministic local parsing with a labelled-grammar (LABELS) regex table, capturing the matched source line as evidence; (2) an LLM pass over the first 15,000 + last 15,000 characters (documents are long, budgets are small); (3) mergeWithLocalMetrics - local values win wherever the label line is unambiguous, because regex on an exact label is more reliable than a model re-typing a number. If the AI is down, step (1) alone still fills the table.

```
// extractionUtils.ts -- the labelled grammar (excerpt)
revenue: /^s*(?:total net sales|total sales|total net revenues?|
  total revenues?|net sales|net revenues?|revenues?)s*[:\u2014-]/i

// extractionAgent.ts -- hybrid merge (excerpt)
extracted = mergeWithLocalMetrics(verified, extractMetricsLocally(content));
```

Numbers are normalised into a consistent shape: percentages as decimals (44.1% -> 0.441), money parsed from '\$383,285 million' style strings, fiscal year detected from 'Fiscal year: 2023' style lines. Every metric can store metric_evidence: the exact line it came from - which is what powers the 'show source' experience and keeps report numbers verifiable.

5.3 Red Flag Agent - riskAgent.ts + analysisUtils.ts

Job: find risks in three deterministic layers plus one AI layer.

- Layer 1 - textual red flags (regex, with severity): qualified/adverse/disclaimer audit opinion or 'except for the effects of' (critical); any going-concern mention (critical); material weakness in internal control / restatement / non-reliance / accounting irregularities (high); debt covenant breach / default / liquidity shortfall / unable to meet obligations (high).
- Layer 2 - metric anomalies: negative margins, collapsing growth, negative free cash flow and similar threshold checks on the extracted metrics.
- Layer 3 - cross-period trends: if another fiscal year of the same company exists, detectFinancialTrendRisks compares them (declining margins, falling revenue, rising debt, negative FCF) and cites BOTH periods as evidence.
- Layer 4 - AI review: the LLM sees excerpts and may add material risks the rules missed - it is asked NOT to duplicate deterministic findings, and its output is stored with source text like everything else.

```
// analysisUtils.ts -- one of four deterministic patterns (excerpt)
{
  pattern: /\b(?:substantial doubt about.{0,100}going[- ]concern|going[- ]concern)\b/i,
  riskType: "Going Concern",
  severity: "critical",
  title: "Going-concern disclosure present",
}
```

Every finding stores risk_type, severity, title, description, source_text (the matched excerpt, capped at 6000 chars) and a recommendation - the Risks page can therefore always show the exact filing language behind a flag.

5.4 Embedding Agent - embeddingAgent.ts

Job: optionally build a semantic search index. For each chunk (batched 32 at a time, each truncated to 8000 chars) it calls any OpenAI-compatible /embeddings endpoint configured by EMBEDDING_BASE_URL / EMBEDDING_API_KEY / EMBEDDING_MODEL, and stores the vector in JSONB with the model name. Search later computes cosine similarity in application memory. If the model name changes, chunks are re-embedded automatically (embeddingModel versioning). If no provider is configured the agent records 'skipped/unavailable' and research uses keyword retrieval instead - nothing else breaks.

```
// llmClient.ts -- provider-agnostic call (excerpt)
const response = await fetch(`${baseUrl}/embeddings`, {
  method: "POST",
  headers: { "content-type": "application/json", authorization: `Bearer ${apiKey}` },
  body: JSON.stringify({ model, input: inputs }),
});
```

This is why the same project runs with Ollama on a laptop (http://localhost:11434/v1) and with a cloud provider on Render - the agent does not care who serves the endpoint.

5.5 Research Agent - researchAgent.ts + analysisUtils.ts

Job: answer questions using ONLY the user's documents, with citations. The flow: (1) decomposeResearchQuestion splits compound questions into retrieval steps (e.g. 'compare margins' becomes one step per company); (2) each step searches the document collection - semantic search when embeddings exist, keyword search otherwise; (3) a relevance filter drops weak matches; (4) buildCitations labels evidence [S1], [S2]... with document name, section, chunk number and an excerpt; (5) the LLM synthesises an answer from the retrieved passages only; (6) validateGroundedResearchAnswer checks that numeric claims in the answer actually appear in the source excerpts; (7) if any AI step fails, the agent returns the verbatim passages clearly labelled as evidence instead of an answer.

```
// researchAgent.ts -- citation construction (excerpt)
citationId: `${index + 1}`

// the answer format the UI renders:
//   ...operating margin was 44.1% [S1]...
//   [S1] Apple FY2023 -- Results of Operations, chunk 2: "..."
```

The 'reasoning' field shows retrieval steps (which searches ran), not hidden chain-of-thought - honest and debuggable. This design is why the demo prompts 'quote the supporting source text' and 'compare X and Y and quote each figure' work so well.

5.6 Report Agent - reportAgent.ts + aiStatus.ts

Job: produce a structured analysis report: Key Financials table, Business Overview, Risk Assessment, Financial Analysis, Outlook and Disclaimer. The Key Financials table is rendered by code from stored metrics (renderKeyFinancialsSection) - the LLM is never trusted to re-type numbers. Narrative sections come from the LLM; if it fails, createLocalReport builds a complete deterministic report and the disclaimer states the reason via describeNarrativeFallbackReason, which maps raw errors (including stale GEMINI_API_KEY-era messages) to clean Groq guidance. Sections are spliced into the markdown with replaceOrInsertMarkdownSection so re-generation cannot corrupt the table.

```
// aiStatus.ts -- clean fallback disclaimer (excerpt)
export function describeNarrativeFallbackReason(error?: unknown): string {
  // maps raw AI errors to: name the real key (GROQ_API_KEY),
  // never leak 'Error:' prefixes or retired provider advice into reports
}
```

Reports are stored in analysis_reports and can be downloaded from the Reports page as text/markdown.

5.7 Benchmark Agent - benchmarkAgent.ts

Job: compare companies on the same scale. It builds a metric profile per company (latest or selected fiscal period), normalises units, then compares revenue/growth, profitability (margins, ROE), leverage/liquidity (debt-to-equity, current ratio) and risk counts. The local engine can rank by ROE and handle missing values with N/A. The AI narrative is instructed to name fiscal periods beside figures, refuse to silently compare mismatched years, and never give buy/sell advice.

```
// benchmarkAgent.ts -- deterministic core (excerpt)
const ranked = [...context].sort((a, b) => Number(b.metrics?.roe || 0) - Number(a.metrics?.roe || 0));
```

5.8 Pipeline Orchestrator + job queue - orchestrator.ts

Job: run upload-time agents in order with failure isolation, and manage a durable job queue. Stage errors are collected, not fatal: a failed extraction does not block risk scanning, and embeddings never block anything. The queue uses an atomic claim (UPDATE document_processing_jobs SET status='processing' WHERE status='queued' AND next_attempt_at <= now()), a 15-minute stale lock so crashed runs can be retried, and a batch size of 5 per worker call.

```
// orchestrator.ts -- failure isolation (excerpt)
try { await extractFinancialMetrics(documentId, content, companyId); }
catch (error) { failures.push(`Extraction Agent: ${...}`); }
try { await scanForRisks(documentId, content, companyId); }
catch (error) { failures.push(`Red Flag Agent: ${...}`); }
// embeddings are optional: a failure only logs a warning
```

The worker endpoint `/api/worker/documents` is protected by `CRON_SECRET` and is triggered by a 15-minute Render cron job (and immediately after upload, so demo speed is instant).

6. Data model (12 tables)

Table	Purpose	Key columns (summary)
users	Accounts	id, name, email, password_hash
research_sessions	Grouping of documents for scoped research	id, user_id, title
companies	Company master data (shared lookup)	id, name, ticker
documents	Uploaded files + processing state	id, user_id, company_id, fiscal_year, content, processing_status, embedding_status
document_chunks	Searchable chunks with optional vectors	id, document_id, content, section, embedding (JSONB), embedding_model
document_processing_jobs	Durable async queue	id, document_id, status, attempts, max_attempts, next_attempt_at, locked_at
financial_metrics	26 extracted metrics + evidence	document_id, fiscal_year, revenue..., metric_evidence (JSONB)
risk_flags	Risk findings with source text	document_id, risk_type, severity, title, source_text, recommendation
analysis_reports	Generated reports	id, user_id, company_id, title, content, disclaimer
chat_messages	Research Q&A history	session_id, role, content, citations (JSONB)
benchmark_comparisons	Saved benchmark runs	id, user_id, payload
agent_logs	Audit trail of every agent action	document_id, agent_name, action, status, details, duration_ms

7. API reference (20 routes)

Method + path	Purpose
POST /api/auth/register	Create account (bcrypt hash)
GET/POST /api/auth/[...nextauth]	Login, logout, session
GET/POST /api/documents	List / upload documents (triggers pipeline)
GET /api/documents/[id]	Document with metrics, risks, chunks, embeddings, activity
POST /api/documents/[id]/reprocess	Re-run the pipeline for a document
POST /api/documents/[id]/embeddings	Start/rebuild semantic index
GET/POST /api/companies	List / create companies
GET /api/metrics	Metric history (cross-period)
GET /api/risks	Risk flags (filterable)
GET/POST /api/sessions, /api/sessions/[id]	Research sessions
POST /api/chat	Research Agent Q&A (citations)
GET/POST /api/benchmark	Benchmark comparisons
GET/POST /api/reports, /api/reports/[id]	Generate and fetch reports
GET /api/dashboard	Dashboard aggregates
GET /api/health	Public AI/agent status (no secrets)
POST /api/worker/documents	Cron worker: claim + process job batch (CRON_SECRET)
POST /api/seed	Demo data seeding (SEED_SECRET)

8. Website guide (every page)

Page	What is on it	What to demonstrate
Home / login / register	Product page and auth forms	Clean registration flow with validation
Dashboard	Counts and recent activity across documents/risks/reports	The whole system at a glance
Documents (+ upload)	Upload panel (drag-drop), document table with processing/embedding status	Upload a PDF and watch the status move to completed
Document detail	Content, extracted metrics with evidence, risk flags, chunks, embeddings state, agent activity log (Build embeddings button)	The agent pipeline explained live - each log row is a real stage
Metrics	Cross-document metric tables and fiscal-period history	26 metrics with numbers pulled from the documents
Risks	Risk cards grouped by severity with source quotes and recommendations	Tesla FY2026 demo: four flags with exact filing language
Companies	Company list with document/metric counts	Multi-company portfolio view
Benchmark	Company selector + comparison table + charts	Apple vs Microsoft vs Tesla on margins and risk counts
Research (chat)	Question box, streamed answers with [S1] citations, retrieval steps	The four demo prompts (value, evidence, comparison, follow-up)
Sessions	Session-scoped research workspaces	Scope a question to one year's documents
Reports	Generated analysis reports + download	Report with deterministic Key Financials table and correct disclaimer
/api/health	JSON status of AI configuration and agents	Live proof of what is configured

9. Setup: local development and cloud deployment

9.1 Local development

```
# prerequisites: Node.js 22, npm, a PostgreSQL database
npm install
npm run db:push          # create tables (uses DIRECT_URL/DATABASE_URL)
npm run dev              # http://localhost:3000
```

9.2 Environment variables

Variable	Purpose	Example / notes
DATABASE_URL	Pooled Postgres connection (app traffic)	postgres://... (Neon -pooler- endpoint)
DIRECT_URL	Unpooled connection (migrations/DDDL)	postgres://... (Neon direct endpoint)
NEXTAUTH_SECRET	Session signing secret	long random string
NEXTAUTH_URL	Public site URL	http://localhost:3000 locally, https://<app> on Render
GROQ_API_KEY	AI for extraction/risk/research/report/benchmark	free key from console.groq.com
GROQ_MODEL	Chat model id	openai/ gpt-oss-120b
EMBEDDING_BASE_URL	Optional: OpenAI-compatible embeddings endpoint	https:// generativelanguage.googleapis.com/v1beta/openai or http://localhost:11434/v1 for Ollama
EMBEDDING_API_KEY	Optional: key for that endpoint	any non-empty value for Ollama
EMBEDDING_MODEL	Optional: embedding model id	gemini- embedding-001 / nomic-embed-text
CRON_SECRET	Protects the worker endpoint	Bearer token used by the cron job
SEED_SECRET	Protects the demo-seed endpoint	Bearer token

9.3 Deployment (Render + Neon)

- 1. Create a Neon project; copy the pooled URL to DATABASE_URL and the direct URL to DIRECT_URL.
- 2. Create a Render Web Service from the GitHub repo: build 'npm ci && npm run build', start 'npm start', Node version pinned by .nvmrc (22).
- 3. Add the environment variables above in the Render dashboard.
- 4. Optional: add a Render Cron Job every 15 minutes: curl -H "Authorization: Bearer \$CRON_SECRET" https://<app>/api/worker/documents
- 5. Optional: UptimeRobot pings the site every 10 minutes to reduce free-tier cold starts.

Known platform facts worth stating out loud: free instances cold-start in about a minute; embeddings work only when a provider endpoint is reachable from the cloud (a laptop's Ollama is reachable only from that laptop); uploads are validated by extension and size, and PDF text extraction rejects scanned/image-only PDFs with a clear, actionable error.

10. Security, reliability and graceful degradation

10.1 Security

- Passwords are hashed with bcryptjs (salted, slow by design) - plaintext passwords never touch the database.
- Authentication via NextAuth with HTTP-only session cookies; every data query is scoped to session.user.id (users cannot see each other's documents, chats or reports).
- All SQL goes through Drizzle ORM (parameterised queries) - no string-concatenated SQL, no injection surface.
- Server-side validation helpers (src/lib/validation.ts) check UUIDs, file extensions and sizes before anything is stored.
- Secrets (GROQ_API_KEY, CRON_SECRET, SEED_SECRET...) live only in server environment variables - the client bundle contains none of them.
- Worker and seed endpoints require Bearer-token secrets, so nobody can drain jobs or seed data anonymously.
- AI error messages are sanitised before display (describeNarrativeFallbackReason) so raw stack traces and stale key advice never leak to users.

10.2 Graceful degradation (what fails safely)

Failure	What the user sees	What still works
Embedding provider missing/down	Embedding status 'unavailable'/'skipped' with explanation	Keyword search research, all other agents
Groq AI down or key invalid	Reports get the deterministic fallback + clean disclaimer	Metrics (regex), risks (regex/thresholds), keyword research evidence
One pipeline stage throws	Document status 'partial' with the failing stage named in activity	Later stages still run
Corrupt/scanned PDF upload	422 with the exact parse reason and what to do	Everything else
Worker never runs	Jobs stay queued and are retried when the cron fires	Documents still saved; processing resumes

11. Testing

The project ships with node:test suites runnable via 'npm test' (node --import tsx --test):

Suite	What it proves
src/lib/agents/ extractionUtils.test.ts	Metric label grammar, number/percent parsing, evidence capture
src/lib/agents/ analysisUtils.test.ts	Red-flag patterns, anomaly thresholds, trend detection, similarity maths
src/lib/agents/ aiStatus.test.ts	AI fallback reasons name the right key; retired-provider advice never leaks
src/lib/agents/ llmClient.test.ts	Request shaping and error mapping for the AI/Embeddings client
src/lib/ validation.test.ts	UUID, file type and size validation
src/lib/ dbConfig.test.ts	Pooled vs direct connection selection
src/lib/ embeddingUiState.test.ts	Embedding status surface (configured/unavailable/failed states)
src/lib/ seedPlan.test.ts	Deterministic demo-seed planning

12. Troubleshooting

Symptom	Cause	Fix
'Could not extract text from this PDF...'	File is corrupt, an error page saved as .pdf, or a scanned image	Re-download/regenerate the file; the error now names the parse reason
Embedding Agent 'skipped/unavailable'	No EMBEDDING_* configured (Groq has no embeddings API)	Set the three EMBEDDING_* variables; click Build embeddings
Embedding Agent 'failed' on Render	EMBEDDING_BASE_URL points at localhost (a laptop-only address)	Use a cloud endpoint on Render; keep Ollama for local dev only
Reports mention a missing/odd key	Stale AI error text from an older build	Current build maps fallbacks to clean Groq guidance; regenerate the report
'fetch first' on git push	Remote and local histories diverged	git pull --rebase, then push; never force-push shared branches
Cold start ~60s on Render	Free tier sleeps the instance	Expected; UptimeRobot pings reduce it

13. Limitations and future roadmap

- Scanned/image PDFs are rejected by design (no OCR pipeline yet) - roadmap: add OCR (e.g. Tesseract) for scanned filings.
- Vectors use JSONB + in-app cosine - perfect for small/medium collections; roadmap: pgvector with an HNSW index for large corpora.
- Semantic features need an external embedding provider; the system stays fully usable without one (keyword mode).
- Figures come from the documents as written - there is no live market-data feed; roadmap: optional price/quote APIs for valuation ratios.
- AI narratives are grounded and validated but never investment advice; the disclaimer in every report says so.
- Roadmap extras: Docker image, CI/CD with test gates, multi-user organisations/roles, streaming chat tokens, multi-language filings, mobile layout.

14. Glossary for beginners

Term	Plain-English meaning
Agent	One module with a single job (read, extract, flag, embed, research, report, benchmark) that the orchestrator calls in order
Chunk	A fixed-size piece of a document (here 1500 characters) stored separately so it can be searched and quoted
Embedding	A list of numbers (vector) representing the meaning of a text, so similar texts have similar vectors
Cosine similarity	A maths formula measuring how close two vectors are (1.0 = identical meaning direction)
RAG	Retrieval-Augmented Generation: search the user's documents first, then answer using only what was found
Grounding	Keeping an AI answer tied to real source text; here numbers must appear in the retrieved excerpts
Deterministic	Code that always produces the same output for the same input (regex, thresholds) - unlike an LLM
ORM	Library that writes SQL for you safely (here Drizzle)
JSONB	A Postgres column type storing flexible JSON (used for vectors, evidence, citations)
Job queue	A table of pending work processed in the background so uploads never wait
Cold start	Delay while a sleeping free-tier server boots up again
Graceful degradation	When one feature fails, the rest keep working in a reduced but honest mode

End of documentation. The presentation deck (PROJECT_PRESENTATION.pdf) turns this material into an evaluator-ready walkthrough with demo scripts and code talking points.